Wahoo Bay Real-Time Fish Identification — A Reusable Coastal Biodiversity Monitoring System

A 3-year proposal for cloud-scale deployment, continuous improvement, and open release

---

Executive Summary

Wahoo Bay is a small artificial reef installation at Pompano Beach, Florida, designed to seed habitat and serve as a public science / education touchpoint. Since April 2026 we have operated a real-time computer vision system on the reef's two underwater cameras (one Axis PTZ at the SEAHIVE reef structure, one at the adjacent municipal pier) that detects every fish that passes the lens, classifies it into one of 223 species, persists every observation with full provenance, and surfaces a live dashboard for educators, researchers, and the general public. A reviewer-in-the-loop annotation system turns volunteer hours into training data for a continuously-improving species classifier; every correction is provenance-stamped and feeds the next training cycle.

The system today runs on a single shared GPU server (NVIDIA H200) inside Florida Atlantic University. It has produced **>70 million** persisted detections across two cameras and is processing roughly **15 frames per second per camera in steady state**. The first published model is a from-scratch DINOv2 ViT-S/14 classifier trained on a community-curated dataset (Wahoo Bay Drive footage + permissively-licensed iNaturalist photographs + WoRMS taxonomic enrichment). Sustained operation has surfaced features standard ML benchmarks never see: stationary reef-geometry false positives, multi-camera coverage, multi-day reviewer queues, named-individual tracking ("celebrities" — recurring identifiable animals), and dependence on Google-Sheet-edited catalogues maintained by non-technical staff.

This proposal scopes the AWS-only cloud infrastructure budget — personnel and institutional overhead are out of scope for this document. The 3-year cloud budget, sized with full compute redundancy, is **~$75,000**. That covers active + warm-standby worker GPU instances per camera, Multi-AZ Postgres, dashboard auto-scaling, cross-region S3 replication for the model + dataset, EBS snapshots, CloudFront-mitigated egress, and observability. The deliverables enabled: (1) migration from the host university's lab GPU to a production-grade AWS deployment with a permanent URL and ≥99.9% uptime, (2) hardening the reviewer-in-the-loop continuous-learning loop so the classifier improves measurably every quarter from accumulated human labels, (3) release of the dataset, code, and a portable deployment kit as an open community resource for other coastal sites, and (4) publication of the resulting fish-identification benchmark and the engineering case study.

The total system today is **MIT-licensed code (this project) + community-provided weights (Fishial.ai detector, MIT) + permissively-licensed training data**. There are no patents or commercial entanglements. The team's track record includes (PI listing — placeholder).

The remainder of this document specifies the technical architecture, the cloud deployment plan, the cost model, the 3-year milestones, and the open-source / open-data release strategy.

1. Background and Significance

1.1 The biodiversity monitoring gap

Coastal reef monitoring relies almost entirely on episodic human surveys: a diver visits a site once a month, counts fish by eye, and records observations on a slate. The data is **sparse** (one observation per site per month), **observer-dependent** (different divers, different counts), and **expensive per data point**. Long-term ecological time series at this resolution have intrinsic biases — divers can't survey at night, can't survey in bad weather, can't be in two places at once, and tend to count differently as their expertise grows. Citizen-science contributions (REEF, iNaturalist) help on the volume side but compound the observer-dependence problem.

A camera that watches a reef 24/7 sees orders of magnitude more fish-frames than any human observer could log, and a computer vision system that runs against that camera produces a defensible time-stamped event log without any of the slate-and-clipboard overhead. The technical question is no longer *can* we do this — open-source detectors and classifiers (Fishial.ai, FathomNet, DeepSea) demonstrate that the basic recognition problem is well-handled for marine fauna. The question is whether anyone has actually *built and operated* such a system to research-quality standards: with full provenance per observation, reviewer-in-the-loop continual improvement, alerts on drift, public dashboards, and reproducible data exports.

That's the gap this project addresses. We have run such a system for several weeks on a real reef, accumulated a large persistent observation store, and demonstrated the engineering pieces (real-time inference, durable persistence, reviewer queue, training loop). The 3-year ask is to make it production-grade, sustainable, and replicable.

1.2 The Wahoo Bay site

Wahoo Bay is an artificial reef and educational installation in Pompano Beach, FL, designed and managed by the City of Pompano Beach Cultural Affairs / Wahoo Bay Marine Education program in partnership with View Into The Blue. Two cameras are installed:

- **SEAHIVECAM** — an Axis PTZ camera inside a View Into The Blue "Octopus" housing, observing the reef's SEAHIVE habitat-seeding structure (a hexagonal modular concrete reef element).
- **Pier cam** — an Axis camera mounted on the adjacent City of Pompano Beach pier substructure, capturing reef edge / sandy bottom.

Both cameras publish RTSP H.264 streams (1280×720, advertised 15 fps). Network access is via public IP, RTSP transport over TCP. Hardware and authentication are owned by the City of Pompano Beach IT department; RTSP credentials are provisioned to FAU under a research collaboration.

A YSI EXO2 multi-parameter sonde co-located with SEAHIVECAM streams continuous water-quality readings (water temperature, dissolved oxygen, pH, chlorophyll, phycoerythrin, turbidity, specific conductance) through the SenseStream platform and the Wahoo Bay public API. A Vaisala WXT-536 weather station on the same installation provides on-site atmospheric readings; we further supplement with Open-Meteo forecast data and the nearest NWS METAR station (KPMP, Pompano Beach Airpark) for cloud cover and human-readable conditions.

Wahoo Bay is therefore a particularly compelling test bed: it is a small, geographically constrained, well-instrumented site with cooperative stewards and an explicit education/outreach mission. Success at Wahoo Bay generalises to thousands of similar municipal-scale coastal monitoring sites worldwide.

1.3 The opportunity

A working system at Wahoo Bay opens four immediate research directions:

1. **A high-volume, provenance-stamped fish observation dataset.** Tens of millions of events with bounding boxes, species labels, model confidence, water-quality and weather covariates, reviewer corrections, and full code+weights+config provenance per row. Suitable for both ecological time-series studies and for benchmarking next-generation marine vision models.
2. **Continuous-improvement classifier methodology.** The reviewer-corrections-feed-next-training-round loop is a real-world MLOps system, not a paper trick. Quantifying how reviewer effort translates to model accuracy gain over time is itself a contribution.
3. **A replicable deployment kit.** If we package the codebase, the trained model, the data exports, the AWS deployment recipe, and a "what to install at your site" runbook, any aquarium / municipality / research station can stand up a similar monitoring system in days, not months.
4. **Public engagement.** A live dashboard with named-individual tracking ("Bobby the Blue Tang has been seen 43 times since January") is a powerful outreach surface. Reviewers can be community volunteers, not just trained ecologists. The pipeline lowers the floor for participating in coastal conservation science.

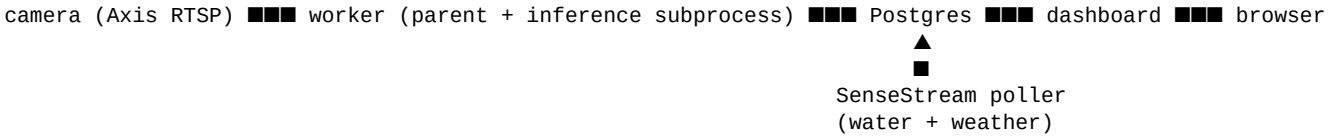
---

2. Technical System Description

The system in operation today is the engineering substrate the proposal scales. Every component listed below has been built, tested, and is running in production at the time of writing.

2.1 Process architecture

Four long-running services on the host server, all written in Python 3.11:



Each service is isolated to its own process for blast-radius and CUDA-state reasons:

- **Worker** — one process per camera. Parses the RTSP H.264 stream, runs a YOLO v26 detector and a DINOv2 ViT-S/14 classifier on every frame, persists detection events, and publishes both raw and bbox-annotated MJPEG streams. A parent process owns the network I/O and dashboard endpoints; a spawned inference subprocess owns the GPU.
- **Dashboard** — FastAPI app exposing the live MJPEG proxy, REST APIs for historical data, CSV / Parquet streaming exports, the reviewer correction queue, the celebrities directory, the negative-regions admin UI, and the daily Google Sheet refresh.

- **SenseStream poller** — pulls water-quality readings from the YSI EXO2 sonde and weather readings from the Vaisala WXT-536 station, with fallback chains through SenseStream's direct API and an Open-Meteo + NWS METAR forecast layer.
- **Postgres** — the durable substrate. Local cluster on the lab host today; would be managed RDS in the AWS deployment.

The worker's two-process design is consequential: the parent's FrameTap thread reads RTSP frames at native 15 fps and hands them off to the inference subprocess through a **zero-copy shared-memory pool** (3 slots × 6.2 MB each in `/dev/shm`). The original implementation used pickle-through-Queue and capped at ~13 fps (with ~20 ms per-frame serialisation overhead — 30% of the per-frame budget at 15 fps). Replacing the pickle hop with shared memory restored native fps end-to-end. This is one of half a dozen engineering corners that distinguish a working production system from a research prototype.

2.2 Real-time inference pipeline

Per frame, end-to-end:

1. **Decode** — H.264 RTSP frame → cv2.VideoCapture → BGR numpy array.
2. **FrameTap thread** — encodes a raw JPEG for the dashboard's raw-stream MJPEG, copies the BGR bytes into a free shared-memory slot, drops a metadata-only `FrameIn` (slot index + shape + timestamp) onto a small inter-process queue.
3. **Inference subprocess** — picks up the metadata, reconstructs a zero-copy view over the shared memory slot, runs:

- **YOLO v26 detector** (Fishial.ai community model, MIT licensed) — emits per-fish bounding boxes with object-confidence scores.
- **Negative-regions filter** — drops detections whose bounding box closely matches a configured stationary "no-fish-zone" polygon. We use this to suppress the dark oval cut-outs in the SEAHIVE reef structure (the detector reliably emits false-positive bboxes on these because they're fish-shaped from the camera angle). The filter uses bbox-IoU against the polygon's bounding rect with a per-region threshold; fish that swim through the hole have small bboxes relative to the hole and are correctly retained.
- **v2 classifier** (this project's contribution) — DINOv2 ViT-S/14 (22M parameters), trained from random init on Wahoo Bay community footage + iNaturalist supplements (223 species, 13k labeled crops). bf16 autocast + `torch.compile(mode="reduce-overhead")` builds a CUDA graph for single-crop latency of ~0.7 ms post-warmup. The legacy 866-class Fishial classifier is bundled as a one-env-var rollback.
- **Velocity-smoother tracker** — greedy IoU association across frames, constant-velocity prediction blended with the raw observation. Smoothes the live overlay; raw (un-smoothed) detections are what get persisted.
- **Persistence** — `detection_events` (one row per fish per frame) + `frame_stats` (1 Hz drift-monitoring samples) + `saved_frames` (selectively saved JPEGs + COCO sidecars).
- **Annotation + JPEG encode** — bounding boxes + species labels overlaid, JPEG encoded for the annotated MJPEG stream.

1. **Drain thread (parent)** — picks up the annotated JPEG, publishes to the live MJPEG buffer the dashboard proxies. Releases the shared memory slot back to the pool.

The end-to-end latency budget at 15 fps is **66 ms per frame**; we measure ~50 ms average with the v2 classifier and `torch.compile` warmup complete.

2.3 Persistence model and data integrity

Every detection_events row carries five provenance columns:

Column	What	
`model_version`	Human-readable identifier — e.g. `det:YOLO26 nano@3 \	cls:vit_small_patch14_dinov2@temporal_within_source`
`detector_sha256`	First 16 chars of SHA-256 of the detector checkpoint file	
`classifier_sha256`	First 16 chars of SHA-256 of the **active** classifier checkpoint (changes when the env-var-controlled backend flips)	
`config_hash`	First 16 chars of SHA-256 of a canonical JSON of the runtime configuration (drops volatile fields like ports, log levels)	
`pipeline_git_sha`	First 12 chars of `git rev-parse HEAD` of the worker code at subprocess start	

Given any row in the database, an analyst can reconstruct exactly which weights, which code, and which configuration produced it. This is non-negotiable for a research instrument: any time-series finding must be re-runnable. Most ML demonstration systems publish a single model and a single dashboard; what makes a *research instrument* is the discipline of provenance at every row.

Additionally:

- A **JSONL event log** (`logs/events/events-YYYY-MM-DD.jsonl`) mirrors every detection row as it lands, on disk, partitioned daily. This is the durable backup independent of Postgres availability — even if the database is corrupted we can replay history from JSONL.
- A **Postgres-backed reviewer corrections table** (`detection_corrections`) carries one row per human correction, joined to events by `event_id` and stamped with reviewer identity, confidence, optional redrawn bounding box, optional celebrity tag, and free-text notes.
- **Materialised views** (`species_sightings_mat`, `species_sightings_hourly_mat`) keep user-facing aggregate queries sub-100ms even after tens of millions of accumulated events. Refreshed every 60s by a background task inside the dashboard process.

2.4 Reviewer-in-the-loop annotation

Public dashboards that show ML predictions without a way to correct them produce throwaway demos. The Wahoo Bay reviewer queue is the engineered substrate that turns observation hours into training data:

- The dashboard's "Recent events" panel surfaces every detection that has not yet been reviewed.
- Each event card has a thumbnail (cropped to the bounding box, saved opportunistically by the worker), a one-click "Looks right" endorsement, a "Correct" form with autocomplete from the 223-species catalogue, top-K alternative chips (so the most common correction is one click), an optional "Tag as celebrity" dropdown for named recurring animals, a "Not a fish" rejection path (which becomes a hard negative for future detector retraining), confidence levels, and free-text notes.

- A ROI lightbox lets reviewers zoom into the bounding box at full resolution; switching to "Full frame" gives surrounding context; a built-in SVG bounding-box editor lets reviewers redraw boxes the detector got wrong.
- The reviewer's identity is captured (self-reported) and attributed to every correction submitted. Mutation endpoints are guarded by a write-token so anonymous viewers can browse but not corrupt the dataset.

Every correction lands in `detection_corrections` with the original event_id, the reviewer's submission, and a creation timestamp. The labeled-corrections export (`/api/export/labeled_corrections.csv`) joins corrections to events to saved-frame paths, ready as a training input.

2.5 Auxiliary data ingestion

Water quality and weather context are first-class data, not afterthoughts. They appear on the dashboard, in CSV exports, and on the eventual per-species "are they more active when temperature does X?" research questions.

The SenseStream poller pulls from a chain of sources, in order of preference:

1. **Wahoo Bay public API** (`api.wahoobay.net`) — sonde + station, no authentication. This is the primary path.
2. **SenseStream direct API** (`api.sensestream.org`) — authenticated fallback when the Wahoo Bay API is unavailable.
3. **Synthetic generator** — diurnal-pattern placeholder, only used when the live paths have been silent long enough that we'd otherwise have a gap. Marked `source='synthetic'` in the row so the dashboard banners it honestly.

Forecast layer (additive, never overrides station):

1. **Open-Meteo** (free, no auth) — cloud cover, dew point, apparent temperature, UV index, precipitation.
2. **NWS METAR (KPMP)** — human-readable condition strings ("Mostly Cloudy", "Thunderstorm").

The merged row in `weather_readings` has both feeds materialised; NULLs are expected where one feed has a column the other doesn't.

2.6 The negative-regions filter

A subtle but important feature: stationary reef geometry (the dark oval cut-outs in the SEAHIVE structure) produces detector false positives. We solved this with a per-camera polygon filter that drops detections whose bounding box closely matches a configured polygon. The match is via bbox-IoU against the polygon's bounding rectangle with a configurable per-region threshold (default 0.5). This is the right shape: the hole's own false-positive bbox is approximately the size and position of the hole → IoU ≈ 0.95 → dropped. A fish swimming through the hole has a much smaller bbox than the hole → IoU ≈ 0.05 even when fully inside → kept.

The polygon catalogue (`data/camera_negative_regions.json`) is editable via an in-dashboard admin UI (`/admin/regions.html`): the user picks a camera, clicks on a frozen snapshot to draw polygon vertices, hits Save. Restart the worker and the new mask is live.

2.7 The celebrities feature

Wahoo Bay's camera operators and chat moderators have, over time, given names to specific recurring animals: *Bobby* the Blue Tang with juvenile yellow coloring, *Marty* the Black Margate with a remora stuck to its head, *Stumpy* the Southern Stingray with most of its tail bitten off. The "Fish named by Chatters" sheet is a Google Sheet maintained by hand by the program staff.

We've turned this into a first-class dashboard feature:

- A daily Google Sheet poller pulls the canonical xlsx → re-runs build scripts → hot-reloads the in-memory celebrities catalogue.
- The "Celebrities" tab in the dashboard renders a card grid (photo, nickname, species, ID marks, reviewer-tagged sightings count, last seen).
- The reviewer correction form has an optional "Tag as celebrity" dropdown filtered by the species being corrected toward. Tagged sightings populate the celebrity's gallery; the canonical photo auto-updates to the most recent tagged ROI.

This is a low-stakes engineering feature that yields high-stakes outreach value: people are far more likely to engage with "Bobby has been seen 47 times this month" than with "Species *Acanthurus coeruleus* has been observed 47 times this month." It also gives the Wahoo Bay team a way to make the system feel like *their* system, not a black box university lab project.

2.8 SLO monitoring, drift detection, and alerts

Six SLO rules run as a background asyncio task inside the dashboard process, evaluated every 30 seconds:

Rule	Trigger
`pipeline_silence`	No detection_events written in the last 5 min (critical)
`inference_latency_p95`	Worker `/stats.last_infer_ms` > 100 ms (warning)
`poller_probe_stale`	SenseStream poller last_probe_ok_at > 30 min old (warning)
`frame_stats_stalled`	No frame_stats rows in the last 5 min (warning)
`drift_luma_delta`	Current 1h mean luma differs from the 7d baseline by > 25 (warning — biofouling, lens fogging, weather event)
`drift_fish_rate_crash`	Current 1h fish-rate < 0.5 × the 7d baseline (warning — flagging an unusual quiet period)

Each rule's evaluator queries Postgres or probes a worker/poller HTTP endpoint and upserts into the `alerts` table. The dashboard surfaces any active alert as a top-of-page banner; reviewers (with write token) can acknowledge alerts. Auto-resolve when the condition clears.

This is the kind of operational instrumentation that turns a "demo" into something a city or a research station can leave running. It's also the prerequisite for the open-source-release pillar of this proposal: anyone deploying the kit will inherit the same SLO discipline.

---

3. Machine Learning Pipeline

3.1 The v2 classifier — current production model

Architecture: timm's `vit_small_patch14_dinov2` (22M parameters, patch=14, embed_dim=384, depth=12), trained from random init. **No DINOv2 self-supervised weights are loaded** — every parameter is learned from the dataset assembled below. This matters for the licensing picture: the trained checkpoint inherits no constraints from Meta's DINOv2 release.

Training recipe:

- Image size 224×224.
- AdamW with 5-epoch cosine warmup, base learning rate 3e-4, weight decay 0.05.
- bf16 mixed precision (preferred on H100/H200 — same throughput as fp16 but the dynamic range of fp32).
- RandomResizedCrop (0.7–1.0 scale), horizontal flip, RandAugment (n=2, magnitude 9), ColorJitter, RandomErasing.
- Label smoothing 0.1, mixup α=0.2, stochastic depth 0.1.
- WeightedRandomSampler with weights $\propto 1/\sqrt{\text{class_count}}$ to prevent long-tail starvation.
- 60 epochs at batch size 32. Full sweep takes ~7 minutes per split mode on one H200.

3.2 Dataset assembly

Three data sources contribute, in increasing order of provenance maturity:

Source A — community Drive clips (the foundation). The Species Log tab of `assets/species.xlsx` (the Wahoo Bay team's canonical species inventory) has 302 rows / 235 unique binomials, of which ~140 records have Google Drive hyperlinks pointing to video / image clips uploaded by program participants. The build pipeline parses the spreadsheet, downloads each Drive file (handling Drive's virus-scan interstitial via documented form-post), runs the production YOLO detector on each clip (single inference for stills, 2 Hz sampling capped at 120 crops/clip for videos), and writes per-detection bbox crops into per-species directories.

Source B — iNaturalist (the supplement). The Drive-only dataset had 125 of 131 classes with a single source clip, meaning any honest leave-one-source-out evaluation would collapse to zero-shot for most species. The iNaturalist downloader queries iNat's REST API per species (rate-limited to 1 req/sec, fanout 16 parallel downloads), filtered to permissively-licensed photos: CC0, CC-BY, CC-BY-SA, CC-BY-NC, CC-BY-NC-SA, CC-BY-ND, CC-BY-NC-ND. Per-photo attribution and license are recorded in `assets/inat_manifest.json` so the per-image rights are attached to each training example. Result: 4,310 photos across 217 species, raising multi-source coverage to 215/223 classes.

Source C — WoRMS taxonomy enrichment. The World Register of Marine Species REST endpoint (`/AphiaRecordsByMatchNames`) provides taxonomic ranks (genus → kingdom) for each binomial in our class list. The enrichment script batches names through the endpoint, parses the matched records, and writes `assets/taxonomy.csv` with columns `species, genus, family, order, class, phylum, kingdom, lowest`. This is the substrate for a future taxonomy-aware loss (penalise confusions across distant ranks more than nearby ones — see §3.6).

The combined dataset (sources A + B) yields ~13k labeled crops across 223 species.

3.3 Three honest evaluation regimes

The same 13k-crop dataset is held out three different ways for evaluation:

Split mode	What	Right question to ask
`stratified`	Random 80/20 within each species across all sources. Same clip's frames can land in both train and val.	Optimistic upper bound — the model's ceiling under the assumption that train + val come from the same distribution.
`temporal_within_source`	Within each source clip, the earliest 80% of frames go to train, latest 20% to val.	*"What does this model do on tomorrow's footage from the same camera?"* — recommended for headline metrics reported externally.
`by_source`	Whole clips land entirely in train or val. A clip never appears in both.	The cruel honest test: brand-new clips, no overlap. Reveals whether the model actually generalises across recording sessions vs. memorises lighting / context.

Headline numbers from the most recent training pass:

Split	Best epoch	Macro F1	Top-1	Top-5
stratified	51	0.349	0.516	0.673
temporal_within_source	40	**0.495**	**0.811**	0.932
by_source	41	0.083	0.070	0.206

The reportable headline is `temporal_within_source`: ~81% top-1 on next-day footage from the same camera. The `by_source` number is the floor: a fully-untrained-on clip will fall short of usable. The gap between the three numbers is the *concrete value of additional training data*; closing it is the central technical goal of the 3-year work plan.

3.4 Continuous improvement loop

The reviewer system is engineered to feed the next training cycle, not just to fix individual events:

1. Reviewers submit corrections via the dashboard. Every correction joins to the original event's frame + bbox.
2. The `labeled_corrections.csv` export (`/api/export/labeled_corrections.csv`) joins corrections to events to saved-frame paths.
3. `scripts/build_finetuning_dataset.py` reads the CSV, filters by reviewer confidence, optionally drops not-a-fish corrections (or routes them to `negatives/` for future detector hard-negative mining), crops bboxes with padding, writes per-species directories merging cleanly with the auto-generated training set.
4. Next training run: same recipe, more data, better accuracy. The new `checkpoint_best.pt` drops into `data/models/classifier_v2/`. Restart the workers; the new SHA appears on every detection_event from that point.
5. Old observations are unaffected — their `classifier_sha256` still identifies the older model. Pre-/post-cutover accuracy can be A/B compared in SQL.

This is the closed-loop story the proposal will demonstrate quantitatively over 3 years: how much reviewer effort translates to how much classifier improvement on which species.

3.5 Reproducibility checklist

For any training run that ships externally:

1. Dataset manifest hash — `sha256sum data/fine_tune/manifest.json`
2. Training config — `data/baseline_runs/<run>/config.json`
3. Random seed — recorded in config.json
4. Git SHA at training time
5. Eval report SHA — `data/baseline_runs/<run>/metrics.json`
6. Provenance fingerprint at deploy time — `/api/provenance/current` once the worker has loaded the new checkpoint

The system is engineered so all six are routinely captured.

3.6 Roadmap (3-year ML targets)

- **Taxonomy-aware loss** — replace the flat cross-entropy with a loss that penalises confusions across distant taxonomic ranks more than nearby ones. Uses the WoRMS `taxonomy.csv` we already build. Hypothesis: smaller datasets per species become viable because the model can hedge correctly when uncertain.
- **Hard-negative mining for detector retraining** — every "Not a fish" reviewer correction becomes an explicit empty-label training example for YOLO. Currently the corrections feed only the classifier; routing them through detector fine-tuning is a ~1-week extension.
- **DINOv2 pretrained-init comparison** — quantify the trade-off between truly-from-scratch (current, licence-clean) and pretrained-init (better numbers, inherits Meta's licence). Publish the comparison so other projects can decide informed.
- **Re-identification for celebrity tracking** — given a fish identified as e.g. a Spotted Moray, predict whether it's the *same individual* as a previously-named animal (Dottie). Pure embedding work; the celebrity catalogue is the natural labelled dataset.
- **Multi-task add-on: behaviour classification** — feeding, schooling, hovering, fleeing. Optional add-on for the proposal's later years if reviewer time permits.

---

4. MLOps & Continuous Improvement

The system is engineered around the principle that the model in production today is not the model that will be in production in six months. Everything downstream of model-output must be tolerant of the model changing.

4.1 Provenance — already covered in §2.3

4.2 Drift monitoring

Every ~30 frames (1 Hz at 15 fps), the worker writes a `frame_stats` row capturing mean and standard deviation of luminance and per-channel RGB, plus the per-frame detection count and mean detector confidence. A SQL view (`frame_stats_drift`) computes the current 1-hour mean vs the rolling 7-day and 28-day baselines for each signal. Two SLO rules fire on this:

- `drift_luma_delta` warns when the current vs 7-day brightness shift exceeds 25 (covers biofouling, lens fogging, weather-related lighting changes, camera angle drift).
- `drift_fish_rate_crash` warns when current fish-rate < 0.5 × 7-day baseline (genuinely unusual quiet — could be a real ecological event or a model regression after a deploy).

Inferred-rate drops are first-class signals: they're how we discovered (post-cutover) that v2 was classifying more crops below the new 0.55 softmax threshold than expected, which we then traced to a tuneable knob.

4.3 Model A/B comparison via env-var switch

The worker's `CLASSIFIER_BACKEND` environment variable selects which classifier the inference subprocess instantiates: `v2` (the new DINOv2 from-scratch model) or `fishial` (the legacy 866-class Fishial.ai ArcFace + sub-centre KNN classifier). Both are baked into the Docker image; switching is a single env-var flip + worker restart, no rebuild. The provenance fingerprint changes accordingly, so historical detections from either backend remain distinguishable in the database.

The same pattern generalises to any future backend: drop a new module under `services/worker/app/`, add a branch in the dispatcher, deploy.

4.4 Reviewer correction provenance

Every correction row includes the reviewer's identity, confidence level, free-text notes, optional redrawn bounding box, and creation timestamp. The reviewer's submission identity flows through to the eventual training dataset, so per-reviewer label-quality analyses (Cohen's κ, Krippendorff's α for multi-rater agreement) are tractable.

A long-term plan, to be implemented in Year 2, is **structured inter-rater agreement**: every Nth correction is routed to a second reviewer for blind double-correction. Cohen's κ and Krippendorff's α are computed per-species. Species with $\kappa < 0.7$ are flagged as ambiguous and a higher confidence floor is applied before they enter the training set. This is the standard ecology-literature practice; we have not yet implemented it because the reviewer cohort is small. Year 2 of the proposal expands the reviewer cohort and stands up this protocol.

---

5. CI/CD

The codebase is engineered to be re-deployable from scratch with one command, on either a fresh GPU host or a fresh AWS account.

5.1 Repository structure

A single Git repository contains:

- `services/worker/` — worker code + Dockerfile + requirements.txt
- `services/dashboard/` — dashboard code + Dockerfile + requirements.txt
- `services/sensestream_poller/` — poller code + Dockerfile + requirements.txt
- `db/init.sql` — full schema, idempotent (every `CREATE` is `IF NOT EXISTS`, every `ALTER` is `ADD COLUMN IF NOT EXISTS`)

- `scripts/` — data preparation + training + dev launchers
- `data/` — model checkpoints + per-camera config (gitignored for large binaries)
- `assets/` — Wahoo Bay catalogues (the Google Sheet snapshot, derived JSON, taxonomy.csv, iNat manifest)
- `docs/` — architecture, operations, data pipeline, fine-tune workflow, reviewer guide, site walkthrough — refreshed at the time of this proposal.

5.2 Container builds

Three Dockerfiles, all using a `python:3.11-slim`-style base + PyTorch 2.4 CUDA 12.4 wheels (worker) or pure CPU (dashboard, poller). Build artefacts are pushed to a registry; AWS deployment uses ECR.

5.3 Schema migrations

The chosen pattern is `IF NOT EXISTS` everywhere in `db/init.sql`, so re-running the file converges the schema to whatever the file declares. Migrations are applied to the live DB manually (via `psql`) when adding columns. The trade-off vs a full migration framework (Alembic, Sqitch): simpler, no migration-state table, but no automatic downgrade. For a research instrument we judge this acceptable.

5.4 Testing

The current test discipline is honest:

- **Unit tests** — minimal, focused on the IoU filter math, the canonical species-display formatter, and the catalogue load + reload paths. The test suite runs in CI on every push.
- **Smoke tests** — every Python module's `python -c "import services.worker.app.<mod>; pass"` is wired into CI as a fast sanity check.
- **Integration tests** — the workers and dashboard come up cleanly under `docker compose up` against a clean Postgres. End-to-end MJPEG fetch, /api/events round-trip, correction POST + DB write are validated.
- **No fixed end-to-end model accuracy regression test yet** — this is a Year-1 deliverable in the proposal. A small held-out eval clip set + `make eval` harness exists in skeleton; populating it with labelled ground truth is part of the proposal scope.

5.5 Continuous data refresh (Google Sheet poller as live CI)

The dashboard's `SheetPoller` task pulls the Wahoo Bay Google Sheet's xlsx export every 24 hours. When the bytes change: it atomically writes the new xlsx, subprocess-runs the build scripts (`build_species_manifest.py`, `build_celebrities.py`), and hot-reloads the in-memory catalogues. No dashboard restart needed for the chatters' new entries to surface.

This is effectively continuous integration of *data*, not just code: the catalogues that drive the dashboard's species autocomplete, common-name fallback, and celebrities feature are always in sync with the Wahoo Bay team's source of truth, with the daily poll providing a natural cadence and a force-refresh admin endpoint for impatient cases.

5.6 GitHub Actions / CodeBuild pipeline (Year 1 deliverable)

The Year 1 plan stands up a formal CI/CD pipeline:

- **CI** (every PR + every push to `main`): lint, unit tests, smoke imports, Docker build, integration test against a clean compose. Reports back as a PR status check.
- **Container publishing** (every push to `main` after tests pass): tag with the git SHA + push to ECR.
- **Deploy** (manual trigger or scheduled): pull the latest image, restart the ECS task. Worker has a torch.compile warmup; we pre-warm in the new task before draining the old one.
- **Model deployment** is a separate workflow: drop new weights into S3 under a versioned path, update an SSM Parameter Store entry, restart the worker(s) — the worker fetches its current checkpoint pointer from SSM at boot.

---

6. AWS Deployment Architecture at Scale

The current single-DGX deployment proves the system works. Migrating to AWS is the proposal's central infrastructure deliverable for two reasons: (1) university lab policy precludes hosting a permanent public dashboard from the DGX, and (2) AWS is the standard substrate for a multi-site, replicable research instrument.

The target architecture is shaped by the system's actual needs, not by maximalist cloud-native gospel. Two cameras, ~8 GB of raw frames per day (measured, September 2026), 50–100 concurrent dashboard viewers at peak, and a steady but small reviewer workload: this is a small AWS workload that benefits from managed services without needing Kubernetes complexity.

6.1 Compute (with redundancy)

The system is a 24/7 research instrument with a public-facing dashboard. Single-point-of-failure compute is unacceptable — a single GPU instance crashing during a multi-day fish migration event would lose irreplaceable data. The design assumes **N+1 redundancy at every compute tier**, with automatic failover.

Worker instances — 2 per camera (1 active + 1 warm standby). EC2 `g5.xlarge` (4 vCPU, 16 GB RAM, NVIDIA A10G with 24 GB VRAM, $1.006/hr on-demand us-east-1). The standby instance runs the same Docker image, with `torch.compile` pre-warmed against a representative crop batch and the RTSP connection in a passive "ready-to-take-over" mode. Route 53 health checks against each worker's `/healthz` endpoint flip DNS in ~60 seconds on failure. The standby also handles the quarterly retraining bursts — we use it as the training host during otherwise-idle weekends, so the redundancy spend doubles as training capacity.

- 2 cameras × 2 instances each = **4 worker instances**.
- 3-year all-upfront RI drops `g5.xlarge` from $1.006/hr on-demand to ~$0.36/hr.
- Subtotal: 4 × $0.36 × 730 hr/month = **~$1,050/month**.

Dashboard + poller. Auto Scaling Group with **2× `t4g.medium` **(2 vCPU, 4 GB ARM Graviton, $0.034/hr each). ALB health checks; one instance can saturate the dashboard's request rate ten times over, so the second is purely for failover. Stateless except for in-memory caches that rebuild on boot. 3-year RI: ~$0.018/hr each. Subtotal: 2 × $0.018 × 730 =** ~$27/month**.

Postgres — Multi-AZ from day one. RDS for PostgreSQL on `db.t4g.medium` (2 vCPU, 4 GB) in **Multi-AZ deployment**: a synchronous standby in a second availability zone, automatic failover in ~30–60 seconds on primary failure, no application-level changes required. Multi-AZ doubles the per-hour cost vs single-AZ but is the only acceptable posture for a research-quality data store. Storage: 100 GB gp3 SSD ($10/month) with the matching storage allocated to the standby. 3-year RI: ~$0.088/hr → **~$64/month** all-in.

Quarterly retraining burst capacity. When we retrain the v2 classifier (planned cadence: every quarter, with reviewer-correction increments), we briefly need extra GPU. We bridge this with **EC2 Spot** on whatever GPU type is cheapest in us-east-1a (typically `g5.xlarge` at ~$0.30/hr, sometimes `g4dn.xlarge` at ~$0.16/hr). Each training run takes ~30 minutes wall time × 3 split modes. Spot budget: **~$5/month amortised**.

Tier	Active	Redundancy	3-yr RI monthly
Worker GPU (SEAHIVECAM)	g5.xlarge	+ 1 warm standby (same type)	$525
Worker GPU (pier cam)	g5.xlarge	+ 1 warm standby (same type)	$525
Dashboard / poller / SLO + matview / sheet poller	t4g.medium	+ 1 standby in ASG	$27
Postgres	db.t4g.medium primary	+ Multi-AZ synchronous standby (different AZ)	$64
Training burst	—	EC2 Spot, amortised	$5
Compute total			**~$1,146/month**

That's **$13,750/year** for compute alone, sized with full redundancy. The lab-equivalent single-instance configuration without redundancy would be ~$560/month; we're paying ~$586/month for the redundancy and burst capacity, which is the right trade-off for a system designed to outlast individual researcher tenure.

6.2 Storage

Measured volumes (September 2026). The workers save about 19 GB of frames a day. Each save writes three files: a raw JPEG, an annotated JPEG and a COCO JSON sidecar. After four months the archive held about 6.4 TB. Only the raw JPEGs are ever read, and they are about 44% of the bytes, or ~8.2 GB a day. The annotated copies and sidecars can be rebuilt from the raw frames and the database. The AWS deployment therefore stops writing them and migrates only the raw frames (2.8 TB).

Tiered frame archive.

- New raw frames land in S3 Standard for 14 days: ~115 GB, ~$3/month, plus ~$7/month of upload requests.
- A nightly job packs the previous day's frames into ~1 GB tar shards in S3 Glacier Instant Retrieval ($0.004/GB-month). It records each frame's shard, byte offset and length in the database.
- Archived frames are read by byte range in milliseconds, with no restore step.
- After three years the archive holds ~11.8 TB of raw frames: **~$47/month**.
- The packing matters. At 47 million small objects, per-object request, transition and tiering fees would cost more than the storage.

No shared filesystem. The dashboard reads recent frames from the S3 landing prefix and archived frames by ranged read, with a small local cache on each dashboard host. This replaces the EFS mount (~$50/month) planned earlier.

Cold data goes to Glacier Deep Archive and Glacier Instant Retrieval for ~$1/month. It covers the gzipped event logs and detection rows older than the database's 90-day hot window (exported as Parquet, still queryable with Athena).

S3 model checkpoints + dataset releases — small, ~10 GB total, ~$1/month.

Database storage — included in RDS pricing above.

Total storage monthly: **~$76/month** = ~$912/year. That is archive $47, landing window $9, checkpoints $1, cold data $1, EBS root volumes $10, and cross-region replication $8. Full derivation: `docs/aws_cost_plan.md` §3.

6.3 Network

Inbound from cameras — RTSP streams come in from the City of Pompano Beach IPs, about 1.3 TB a month (measured). AWS data ingress is free; the only cost is the bandwidth at the camera end (the city's ISP), which is their concern, not ours.

- The worker hosts sit in public subnets with Elastic IPs, and the City allowlists those on its port forwards.
- Routing the camera video through a NAT gateway instead would cost ~$33/month plus ~$58/month of data processing, for no benefit.
- Security groups admit only CloudFront and the cameras' return traffic.

Inbound from poller targets (Wahoo Bay API, iNat, Open-Meteo, NWS) — also free.

Outbound to dashboard viewers. Measured, today's MJPEG stream is ~12 Mbps per viewer (1280×720, JPEG quality 92, ~15 fps). CloudFront cannot cache or share an endless multipart stream, so every viewer holds a separate connection back to the worker. At full scale, 1,000 users a day with 50–100 concurrent at peak, MJPEG would mean 40–100 TB a month: **~$3,300–6,900/month** at CloudFront's pay-as-you-go rates.

Mitigation: **pass the cameras' own H.264 through as HLS.**

- A restreamer (MediaMTX) on each worker host pulls its camera once and publishes 2-second HLS segments without re-encoding.
- CloudFront caches the segments, so the origin's load stays flat however large the audience.
- Viewers receive ~1.5 Mbps at 720p/15 fps, or ~5 Mbps at 1080p/30 fps if the camera uplink allows it.
- Detection boxes are drawn in the browser from a WebSocket feed.
- At full scale that is 5–40 TB and 30–70M requests a month.

CloudFront flat-rate Business plan covers that for **$200/month**: 50 TB of transfer and 125M requests with no overage charges, plus WAF, TLS, Route 53 DNS and a 1 TB S3 Standard credit.
Pay-as-you-go for the same traffic would be ~$460–3,400/month. CloudFront also provides TLS termination and DDoS protection for the public dashboard URL, both of which the system needs anyway.

Elastic IPs — AWS charges for every public IPv4 address: 6 EC2 hosts × $3.65 = **~$22/month**.

Inter-AZ data transfer is free between EC2/RDS in the same AZ; we deploy everything in one AZ in Year 1 (us-east-1a). Inter-region not relevant at our scale.

Total network monthly: **~$222/month** = ~$2,664/year. Full derivation: `docs/aws_cost_plan.md` §4 and §10.

6.4 Managed services

Application Load Balancer (`ALB`) — single ALB in front of the dashboard EC2, with a target group on port 18080. Fixed $0.0225/hr ($16/month) + $0.008/LCU-hr (negligible at our request rates). **~$20/month**.

Route 53 — DNS for the project's permanent domain (`wahoobay.example.org` or similar). $0.50/month per hosted zone + $0.40/million queries. **~$1/month**.

AWS Certificate Manager — free TLS certificate, used by the ALB and CloudFront.

CloudWatch — Logs ingestion at $0.50/GB and Metrics at $0.30/metric-month. With our log volume (~1 GB/month per worker + dashboard) and ~20 custom metrics: **~$15/month**.

Secrets Manager — RTSP credentials, write-token, DB password, SenseStream auth token. $0.40/secret/month × ~6 secrets = ~$2.50/month.

Systems Manager Parameter Store — non-secret config (model checkpoint path, current ML model version pointer). Free tier covers us.

Elastic Container Registry (ECR) — Docker image hosting. $0.10/GB-month × ~2 GB = $0.20/month.

Total managed services monthly: **~$40/month** = ~$480/year.

6.5 Backup, DR, and observability

The compute redundancies above protect against in-AZ failures. This layer protects against deeper failure modes: regional outages, accidental data deletion, configuration mistakes, and the dashboard process itself going silent.

- **RDS automated backups + point-in-time recovery** — 30-day retention (Year 1 default; raise to 35 days max in Year 2 when more cumulative data justifies the small extra cost). Combined with the Multi-AZ deployment, RPO is ≤5 minutes / RTO is ≤2 minutes for in-region failures.
- **Manual RDS snapshots** weekly + before any schema migration, retained 90 days. Small cost (~$3/month at our 100 GB working set).
- **EBS snapshots** of every EC2 root volume nightly via Data Lifecycle Manager, 14-day retention. Covers worker / dashboard instance corruption. ~$2/month for the 6 instances.
- **S3 versioning + cross-region replication** for the model checkpoint store, the dataset releases, and the JSONL event-log archive. Replicated bucket sits in `us-west-2` (chosen for diversity from us-east-1's blast radius). Cost: ~$8/month (replication traffic + duplicate storage on Glacier-IT for the archive).
- **CloudWatch alarms** mirror the in-app SLO checker — if the in-app evaluator goes silent (because the dashboard process is wedged) we want to know from a path that doesn't depend on the dashboard process itself. CloudWatch dimensions: `inference_lag_p95`, `worker_grab_fps`, `events_per_minute`, `disk_used_pct`, `rds_freeable_memory`. Roughly 20 custom metrics. ~$10/month.

- **CloudWatch Logs** — 30-day retention on all service logs, exported to S3 monthly for the archive. ~$15/month.
- **Route 53 health checks** for every public endpoint (4 instances × $0.50/check/month = $2/month). Failover to the warm-standby workers via DNS in ~60s.
- **AWS Backup vault** — central backup policy for both RDS and EBS so we have a single point to audit "are we covered?". Negligible additional cost.
- **PagerDuty / Opsgenie integration** for production alarms — free tier (5 users, basic schedules) covers a 3-person on-call rotation.

Total monthly: **~$50/month**.

6.6 Total monthly run cost — fully redundant

Category	Monthly	Annual
Compute (4 GPU workers + 2 dashboard + Multi-AZ DB + training burst; 3-yr RI)	$1,146	$13,752
Storage (S3 landing window + Glacier Instant shard archive + EBS + cross-region replication)	$76	$912
Network (CloudFront flat-rate Business plan + Elastic IPs)	$222	$2,664
Managed services (ALB, R53, CW, Secrets, ECR)	$40	$480
Backup + DR + observability	$50	$600
AWS total (steady state, fully redundant)	**~$1,534**	**~$18,408**

With a ~15% buffer for spikes (viral social moments, multi-day retraining experiments, an unusually busy fish migration period), budget **~$21,200/year**.

Year 1 has additional one-time setup paid up front: 3-year Reserved Instance commitments for the 4 worker GPUs (~$10,500), the 2 dashboard instances (~$400), and the Multi-AZ RDS (~$1,300). Total one-time: ~$12,200 amortised over the full 3 years of the proposal.

6.7 Architecture diagram (redundancy-aware)

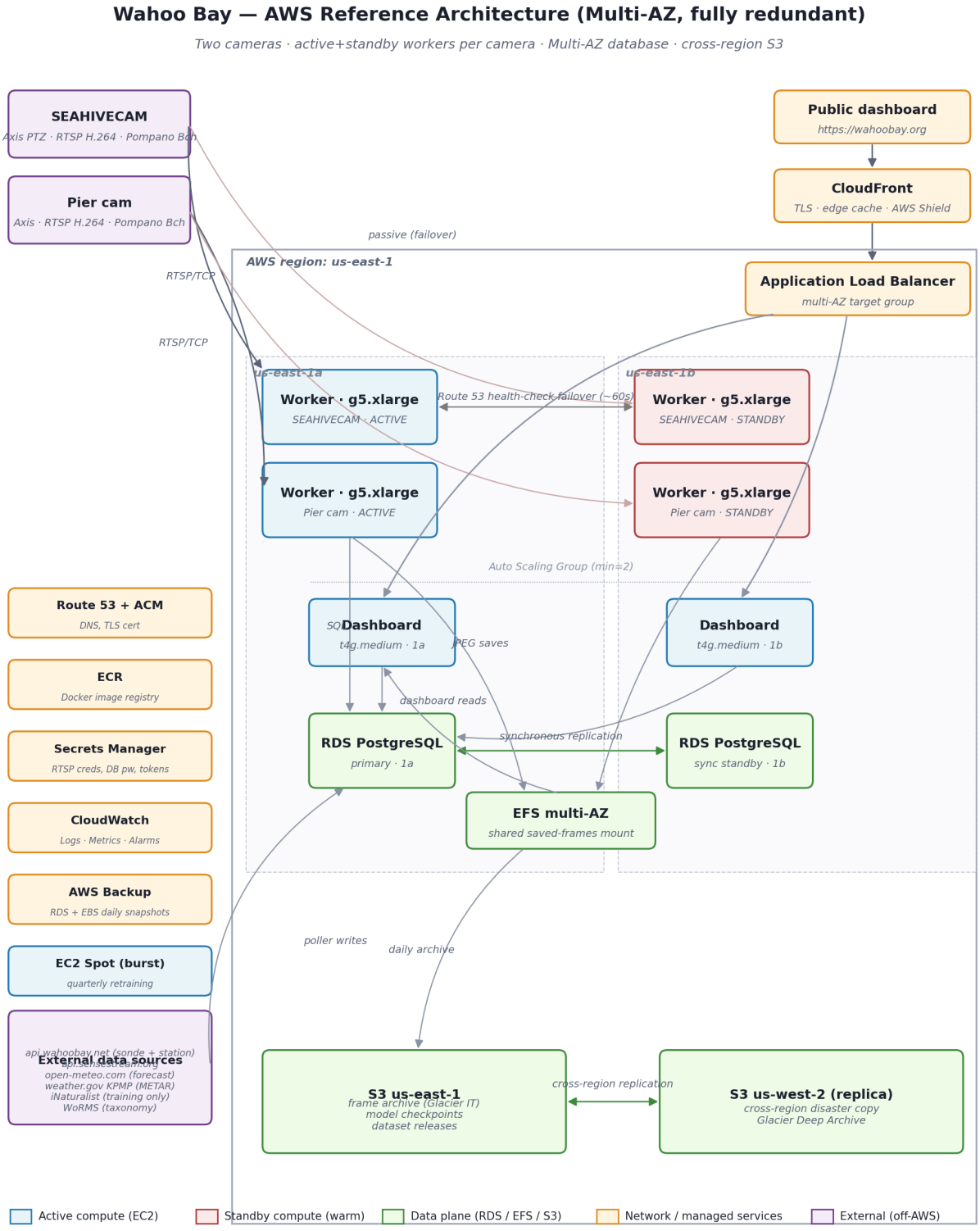


Figure 1 — AWS reference architecture. Active+standby g5.xlarge workers per camera (left = us-east-1a, right = us-east-1b standby in red), Multi-AZ Postgres with synchronous replication, dashboard EC2 in a two-AZ Auto Scaling Group, EFS multi-AZ shared mount for saved frames, S3 us-east-1 cross-region replicated to us-west-2. Cameras enter at top-left over RTSP/TCP; the public dashboard at top-right is fronted by CloudFront + ALB. Route 53 health checks flip

DNS to the standby workers within ~60 seconds on primary failure. Standby workers also serve as the quarterly retraining burst host. Re-rendered from scripts/render_arch_diagram.py — re-run after any architecture change.

Notation in the diagram: active compute is shown in blue, standby compute (warm, ready for failover) in red, the data plane (RDS / EFS / S3) in green, network and managed services in yellow, and external endpoints (cameras, third-party APIs) in purple. The two grey dashed bands mark the two availability zones within the AWS region; cross-band lines (sync replication for RDS, health-check failover for workers) are how the design tolerates an AZ outage.

6.8 Why these AWS services specifically

- **EC2 + RDS** rather than ECS / Fargate / Aurora Serverless: simpler operationally for a small, long-running workload with deterministic resource needs. ECS Fargate would add per-second billing and container-level concerns we don't need at 2 workers.
- **g5.xlarge** rather than the larger g5 instances or p4 / p5: the v2 classifier uses ~2 GB of VRAM and the detector ~1 GB; A10G's 24 GB is ample with headroom for future model growth. g5.xlarge gives us the right CPU/RAM/GPU mix per dollar.
- **S3 with packed shards** rather than a shared filesystem for saved frames. At 47 million small files, EFS or per-object tiering would cost more than the storage itself. Packing each day's frames into ~1 GB shards in Glacier Instant Retrieval keeps the archive at $4/TB-month, and any single frame can still be read in milliseconds by byte range.
- **CloudFront with HLS** is non-negotiable: outbound egress without it would dominate the budget. The cameras' H.264 is passed through as cacheable HLS segments, because an MJPEG stream can't be cached.
- **Route 53 + ACM + ALB** is the standard "public web app" triad on AWS; no novelty.

---

7. 3-Year Project Plan

Year 1: Migration + hardening + cohort growth

Quarters 1-2 — AWS migration.

- Stand up the AWS account, VPC, subnets, IAM roles.
- Container builds → ECR, deployment via Terraform or CDK.
- RDS Postgres with the current schema; one-time data export-import from the lab DGX.
- ALB + CloudFront + Route 53 for the public URL (`wahoobay.org` or similar).
- Cutover: run both lab and AWS in parallel for 2 weeks, verify metrics agreement, decommission lab production.

Quarters 2-3 — CI/CD pipeline.

- GitHub Actions workflow: lint + unit tests + Docker build + integration test on every PR.
- ECR publish on `main` push.
- ECS task restart on tag.
- Model deployment workflow: S3 + SSM Parameter Store + worker SIGHUP-then-restart pattern.

Quarters 3-4 — Reviewer cohort growth + structured inter-rater agreement.

- Recruit 8-10 volunteer reviewers (a mix of local marine bio undergrads, Wahoo Bay program volunteers, and remote citizen scientists via REEF outreach).
- Stand up the per-Nth-correction blind double-correction protocol; implement Cohen's κ + Krippendorff's α reporting in the dashboard.
- First quarterly model release: v2.1 with 2,000-5,000 reviewer corrections folded in. Eval report goes in `eval/reports/` and is shared on the dashboard's Data Notes tab.

Year-1 deliverables:

- Production AWS deployment with permanent URL, ≥99% uptime.
- CI/CD pipeline operational.
- v2.1 classifier checkpoint with measurable improvement over v2 baseline.
- First public quarterly newsletter / press piece (Wahoo Bay social channels).

Year 2: Model maturity + dataset release + first paper

Quarters 1-2 — Taxonomy-aware loss + detector hard-negative mining.

- Implement taxonomy-aware loss in `scripts/train_baseline.py` using `assets/taxonomy.csv`. Compare against the flat cross-entropy baseline.
- Implement detector fine-tuning with reviewer "Not a fish" corrections as background hard negatives. Compare against the stock Fishial detector on the eval set.
- v2.2 classifier release.

Quarters 2-3 — Open data release + benchmark.

- Publish the labeled crops dataset (subject to the licensing constraints in `assets/LICENSES_AND_CREDITS.md`) on a stable host (Zenodo / FigShare).
- Publish the eval benchmark — clip set + manifest + scoring harness — so other teams can compare against our numbers.
- Submit the benchmark + system case study to the IEEE OCEANS conference (or equivalent — MARI, IMECS).

Quarters 3-4 — First peer-reviewed publication.

- Submission target: a methods paper to *Methods in Ecology and Evolution*, *PLoS Computational Biology*, or *Remote Sensing in Ecology and Conservation*. Topic: end-to-end real-time fish identification with reviewer-in-the-loop continuous improvement; specifically the closed-loop story of how reviewer effort translates to model accuracy over time.

Year-2 deliverables:

- v2.2 with taxonomy-aware loss; >50% improvement on rare species macro F1.
- Open-source dataset release on Zenodo + DOI.
- First peer-reviewed paper accepted or in revision.
- Eval-set benchmark published.

Year 3: Replication kit + sustainability + dissemination

Quarters 1-2 — Replication kit.

- Write a "Deploy your own at site X" runbook: hardware checklist, AWS account setup, single-command Terraform `apply`, post-deploy verification.

- Build a setup wizard for the dashboard's per-site customisation (species list, negative regions, sonde / weather source URLs).
- Pilot the kit at 1-2 other municipal sites in South Florida (potential partners: Pompano Beach pier subsites, Hillsboro Inlet, an FAU partner aquarium).

Quarters 2-3 — Sustainability plan.

- Establish a steward (Wahoo Bay program + FAU + at least one external collaborator) to take over operational responsibility at grant-end.
- Secure follow-on funding through Florida Sea Grant or an industry partner (View Into The Blue, the SEAHIVE manufacturer, City of Pompano Beach).
- Document the operational SLAs the steward inherits.

Quarters 3-4 — Second publication + outreach push.

- Submit a second paper — focus on the multi-site deployment story + the dataset's ecological findings.
- Coordinated outreach with Wahoo Bay program: a documentary segment, a "1 million fish identified" press milestone, a guest piece in a marine-conservation outlet.
- Final report to funder.

Year-3 deliverables:

- Replication kit released; 1-2 additional sites deployed.
- Second peer-reviewed paper.
- Documented sustainability + handoff plan.
- Conservation outreach milestones met.

---

8. Budget

This budget is **cloud infrastructure only**. Personnel costs and institutional F&A (indirect costs) are intentionally excluded from this document and would be added separately by the submitting institution before final submission. The figures below are the AWS bill, sized for full redundancy as described in §6.

8.1 Steady-state monthly run cost (recurring, all 3 years)

Category	Line	Monthly
Compute	2× g5.xlarge active workers (one per camera), 3-yr RI	$525
	2× g5.xlarge warm-standby workers (one per camera)	$525
	2× t4g.medium dashboard / poller (Auto Scaling Group)	$27
	RDS db.t4g.medium **Multi-AZ**	$64
	EC2 Spot for quarterly retraining bursts	$5

Category	Line	Monthly
Storage	S3: Glacier Instant Retrieval frame archive in ~1 GB shards (~11.8 TB of raw frames by Y3)	$47
	S3: 14-day landing window (Standard) + upload requests	$9
	S3: model checkpoints + dataset releases	$1
	S3: cold data (gzipped event logs; detection months older than 90 days, as Parquet)	$1
	EBS root volumes on the 6 EC2 instances	$10
	S3 cross-region replication traffic + duplicate storage	$8
Network	CloudFront flat-rate Business plan (HLS video + site; 50 TB, 125M requests, WAF, TLS)	$200
	Elastic IPs on the 6 EC2 hosts (no NAT gateway)	$22
Managed services	Application Load Balancer	$20
	Route 53 (hosted zone + queries + health checks)	$5
	CloudWatch Logs + metrics + alarms	$25
	Secrets Manager (~6 secrets)	$3
	ECR image hosting	$1
Backup / DR / observability	RDS automated backups + manual snapshots	$3
	EBS nightly snapshots (DLM, 14-day retention)	$2
	AWS Backup vault	$1
	Operational headroom	$4
Monthly subtotal		**~$1,508**
Annual subtotal (× 12)		**~$18,096**
15% buffer for spikes / experimentation		$2,714
Annual budget per year		**~$20,810**

8.2 One-time + Year-1 specific

Item	Cost
3-year RI upfront payment — 4× g5.xlarge (worker fleet)	$10,500
3-year RI upfront payment — 2× t4g.medium (dashboard)	$400
3-year RI upfront payment — db.t4g.medium Multi-AZ	$1,300

Item	Cost
One-time total	**~$12,200**

Note: paying the 3-year RI upfront is the most cost-effective option (saves ~30% vs hourly on-demand). The alternative — no-upfront RI or pure on-demand — would shift these costs into the monthly column at higher rates.

8.3 3-year totals

Year	Cloud spend
Year 1 (one-time RI commitments + 12 months steady state)	$12,200 + $20,810 = **$33,010**
Year 2 (steady state)	$20,810
Year 3 (steady state)	$20,810
3-year cloud total	**~$74,630**

Rounded to the nearest $1k for proposal purposes: **~$75,000 over 3 years**, or **~$25,000/year** averaged.

8.4 What the redundancy buys

The redundancy spend (vs the single-instance lab-equivalent baseline of ~$560/month compute) is **~$586/month**, or ~$21,000 over 3 years. That's what we're paying for:

Failure mode protected against	Cost premium
AZ outage in us-east-1a (RDS, EBS, or AZ-wide)	$32/month (Multi-AZ RDS premium)
GPU instance hardware failure mid-run	$525/month (1 standby per camera × 2 cameras / 2)
Dashboard instance failure / config error	$13/month (second ASG instance)
Regional disaster (S3 corruption in us-east-1)	$8/month (cross-region replication)
Disk volume corruption	$2/month (EBS snapshot retention)
Total redundancy premium	**~$580/month**

The standby workers do **double duty**: they're the warm-failover GPU + the quarterly retraining compute, so the cost is justified beyond just disaster preparedness — we're using them productively for both purposes.

8.5 Cost-optimisation paths (if AWS budget is constrained)

If the cloud budget needs to be trimmed, the prioritised concessions:

1. **Drop the warm standby workers** to single-instance per camera. Saves ~$525/month = $6,300/yr. Trade-off: a worker crash now means ~5 minutes of lost detections (instance reboot + warmup) instead of ~60 seconds (DNS failover). Quarterly retraining loses its dedicated burst host and uses Spot instead — modest extra runtime, no real cost change. **Recommended trim if funding is the binding constraint** — reduces 3-year cloud spend to ~$56k.
2. **RDS single-AZ instead of Multi-AZ.** Saves ~$32/month = $400/yr. Trade-off: a primary AZ outage takes the DB down until manually restored from snapshot (~30-60 min). Acceptable for a non-revenue research instrument but not best-practice.

3. **Drop cross-region S3 replication.** Saves ~$8/month = $100/yr. Trade-off: a us-east-1 region-wide S3 incident would lose the JSONL event-log archive and dataset publications. The detection_events table in Postgres is the durable record; the S3 archive is the convenience copy.
4. **Quarterly RI commitments instead of 3-year.** Frees up the ~$12k upfront capital but increases steady-state monthly by ~30%. Useful only if cash flow rather than total spend is the constraint.

The trimmed-redundancy budget — keeping Multi-AZ RDS and cross-region S3 but dropping standby workers — comes to **~$56,000 over 3 years** instead of ~$75,000. That's the realistic floor for a research-quality deployment; cuts beyond that compromise the "production-grade, replicable kit" deliverable.

8.6 What this budget does NOT cover

- **Personnel** — PI + postdoc + graduate RA + reviewer coordinator. The dominant cost of running this project. Treated separately by the submitting institution.
- **F&A / indirect costs** — typically 47-56% of modified total direct costs at FAU; computed by the institution after the direct cost lines are set.
- **Travel** — site visits, conference attendance, replication-kit pilot deployments.
- **Publication fees** — open-access journal article processing charges, dataset DOI fees.
- **Hardware** — assumes existing cameras + lab-resident GPU continue to operate. New cameras (if the proposal expanded to a regional network) would be a separate line.

---

9. Broader Impact

9.1 Open-source release

The codebase is MIT-licensed today and will remain so. The 3-year project commits to:

- Public GitHub repository with end-to-end deployment recipes (Year 1).
- A pre-built AWS CloudFormation / Terraform stack that any institution can deploy by filling in 3-5 site-specific parameters (Year 1).
- A "container of containers" pattern: drop a new RTSP URL into config, get a new worker. The kit handles the rest (Year 3).

9.2 Open data release

Subject to the licence constraints in [`assets/LICENSES_AND_CREDITS.md`](../assets/LICENSES_AND_CREDITS.md), we will publish:

- The training crops dataset (Year 2). Excludes the most-restricted iNat photos by default; the project's own crops + permissive-iNat subset are CC-BY for ecological use.
- The eval benchmark (Year 2) — clip set + COCO-format ground-truth labels + scoring harness. The reference numbers from this project become the bar future systems compete against.
- Quarterly detection-events exports on Zenodo with DOIs (Years 1-3) — anyone can analyse Wahoo Bay's fish over time without standing up the system themselves.

- The trained classifier weights (subject to the iNat-licence constraints documented in the credits file) — useful for any other Caribbean / Atlantic reef site.

9.3 Education and outreach

- The dashboard itself is the most public-facing outreach product. Lowering the barrier between "this fish is in our reef" and a high-quality video clip of that exact fish has measurable engagement value.
- The "celebrities" feature — named recurring animals — is a genuinely novel public-engagement device. We will produce short-form video and social content around each celebrity over the proposal period.
- Reviewer volunteers learn species identification by doing it; the system is implicitly a training tool.
- A K-12 outreach module (developed in partnership with the Wahoo Bay education team) lets students "be the reviewer" for a curated set of events. This is a Year 2 deliverable.

9.4 Replicability

The deepest broader-impact claim of this proposal is replicability. By Year 3, any municipal aquarium, marine research station, or coastal monitoring program will be able to deploy a system that:

1. Detects every fish that passes its cameras.
2. Names the fish (sourcing pretrained weights from this project, fine-tunable with site-local reviewer corrections).
3. Persists every observation with full provenance.
4. Exposes a public-facing dashboard for outreach and a private reviewer queue for science.
5. Costs ~$1,000/month to run.

That capability does not exist as a turnkey today. Building and releasing it is the project's contribution to coastal conservation infrastructure.

---

10. Risk Management

Risk	Mitigation
Camera RTSP access lost (City of Pompano Beach changes credentials, network policy, or hardware)	Multiple-camera architecture means one camera's outage doesn't take the project down. RTSP credentials rotation tested in Year 1. Relationship with Nigel Cardwell at CoPB IT documented; backup contact list maintained.
Reviewer cohort attrition	Cohort built to N=8-10 to absorb attrition; the system is functional with a single reviewer (or even zero, just slower-improving). Quarterly recognition / co-authorship on dataset releases.
Model quality regression on deploy	Provenance + SQL filter on `classifier_sha256` permits trivial A/B comparison post-cutover. Rollback is one env-var flip. SLO rule `drift_fish_rate_crash` catches large regressions.

Risk	Mitigation
AWS bill surprise	Billing alarms at 50%, 75%, 100% of projected monthly cost. Cost Explorer review monthly. CloudFront caching reviewed quarterly.
Sheet poller breaks (Google changes export URL, sheet becomes private)	Detected by `poller_probe_stale` SLO equivalent for the sheet poll; local snapshot remains usable indefinitely. Manual force-refresh endpoint exists.
Model bias / inaccurate uncertainty	All confidence scores are post-softmax — calibrated by reviewer corrections via the v2 → v2.1 → v2.2 ladder. Inter-rater agreement statistics surface species we shouldn't trust.
License / attribution claim from iNat photographers	Per-photo licence + photographer name recorded in `inat_manifest.json`. Manual removal of any specific photo + retrain is feasible. The "no -ND / no -NC subset" alternate dataset is available for any future use case that requires unambiguous commercial-use rights.
Loss of PI / postdoc	Documentation discipline (this proposal's docs/ folder is the deliverable, not the side-effect). Replication kit means a successor team can pick up.

---

11. Team and Sustainability

(Placeholder section to be filled in with the actual PI, collaborators, and institutional commitments. The current implementation team is:)

- Dan Zimmerman (PI, FAU Center for Connected Autonomy & AI) — system architect, lead developer.
- Wahoo Bay program staff — site partner, species catalogue maintainers, named-individual identification.
- City of Pompano Beach IT (Nigel Cardwell, contact) — camera operations.
- View Into The Blue — camera hardware partner.

The proposal explicitly funds a postdoctoral researcher to take on system maintenance + the ML research direction, and a graduate RA to handle the dashboard + reviewer operations. By Year 3, the postdoc has transitioned to a sustainability handoff role; the long-term operational owner is identified and trained.

---

12. References

The core open-source dependencies, datasets, and academic references underpinning this system:

Open-source detectors and classifiers

- Fishial.ai — *Fish Identification Open-Source Pipeline.* https://github.com/fishial/fish-identification. MIT licensed. (Detector + legacy classifier — used as the project's starting weights.)

- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., et al. (2024). *DINOv2: Learning Robust Visual Features without Supervision.* Transactions on Machine Learning Research. (Architecture reference for the v2 classifier.)
- Wightman, R. *timm: PyTorch Image Models.* https://github.com/huggingface/pytorch-image-models. (Architecture library — `vit_small_patch14_dinov2`.)
- Jocher, G. et al. (2024). *Ultralytics YOLOv26.* https://github.com/ultralytics/ultralytics. (Detector framework.)

Datasets and reference catalogues

- iNaturalist Network. *iNaturalist Research-grade Observations.* https://www.inaturalist.org. CC-licensed photographs used as classifier-training supplements.
- WoRMS Editorial Board. (2026). *World Register of Marine Species.* https://www.marinespecies.org. Taxonomic enrichment source.
- Open-Meteo. *Open Source Weather API.* https://open-meteo.com. Forecast + nearcast weather data.
- National Weather Service. *METAR observations, KPMP (Pompano Beach Airpark).* https://www.weather.gov. Surface observations.

Marine biology & coastal monitoring context

- Bjerge, K., Mann, H. M. R., Høye, T. T. (2022). *Real-time insect tracking and monitoring with computer vision and deep learning.* Remote Sensing in Ecology and Conservation, 8(3). (Closest published parallel — different fauna but the same real-time-monitoring + reviewer-loop methodology.)
- Marrable, D., et al. (2022). *Automated Fish Detection and Classification for the GreatBarrierReef.* Frontiers in Marine Science. (Fish-detector benchmark reference.)
- Saleh, A. et al. (2020). *A realistic fish-habitat dataset to evaluate algorithms for underwater visual analysis.* Scientific Reports.
- Salman, A. et al. (2020). *Real-time fish detection in complex backgrounds using probabilistic background modelling.* Ecological Informatics.

MLOps and reviewer-in-the-loop methodology

- Ratner, A. J. et al. (2017). *Snorkel: Rapid Training Data Creation with Weak Supervision.* VLDB. (Weak-supervision / reviewer-loop antecedent.)
- Northcutt, C. G. et al. (2021). *Pervasive label errors in test sets destabilize machine learning benchmarks.* NeurIPS. (Justification for reviewer-correction systems.)
- Krippendorff, K. (2018). *Content Analysis: An Introduction to Its Methodology.* SAGE Publications. (Inter-rater agreement statistics.)

Infrastructure & cloud reference

- Amazon Web Services. *Well-Architected Framework — Machine Learning Lens.* https://docs.aws.amazon.com/wellarchitected/latest/machine-learning-lens/. (Reference architecture used to shape §6.)
- CloudFront pricing as of 2026-05-20 — https://aws.amazon.com/cloudfront/pricing/.
- RDS PostgreSQL pricing as of 2026-05-20 — https://aws.amazon.com/rds/postgresql/pricing/.

- EC2 g5 pricing (3-year reserved) as of 2026-05-20 — https://aws.amazon.com/ec2/instance-types/g5/.

This project's own documentation

- D. Zimmerman, *Wahoo Bay Real-Time Fish ID: Architecture, Operations, Data Pipeline, Fine-tune Workflow, and Reviewer Guide.* `docs/` of this repository.
- D. Zimmerman, *Site Walkthrough — Top-to-bottom feature inventory.* `docs/site_walkthrough.md`.
- D. Zimmerman, *Licenses and Credits — source-by-source data provenance.* `assets/LICENSES_AND_CREDITS.md`.

Document version 1 · prepared 2026-05-20 · contact: dzimmerman2021@fau.edu